IFT-1004 - Algorithmique et programmation

Vers l'examen Intra

Département d'informatique et de génie logiciel
Université Laval

Table des matières

1 Module 1 : Présentation de la programmation **4**
1.1 Algorithme 4
1.1.1 Définition 4
1.1.2 Distinction entre algorithme, pseudo-code et programme 4

2 Module 2 : Les algorithmes et les instructions procédurales **5**
2.1 Pseudo-code 5
2.2 Trace d'un algorithme 5
2.3 Les variables en programmation 5
2.3.1 Caractéristiques d'une variable 5
2.3.2 Types de base 6
2.4 L'opération d'affectation 6
2.5 Entrées et sorties 6
2.5.1 Entrées (au clavier) 6
2.5.2 Sorties (à l'écran) 6
2.6 Opérateurs arithmétiques 6
2.6.1 Opérateurs binaires 6
2.6.2 Opérateurs unaires 6
2.7 Les expressions booléennes 7
2.7.1 Type Booléen 7
2.7.2 Opérateurs de comparaison 7
2.7.3 Opérateurs logiques 7
2.8 Instructions conditionnelles 7
2.9 Les boucles 7
2.9.1 Boucle « tant que » 7
2.9.2 Boucle « pour tout » 8

3 Module 3 : Les bases de la programmation et du langage Python **9**
3.1 Rôle de l'interpréteur 9
3.2 Code source versus code machine 9
3.3 Langage interprété vs compilé 9
3.3.1 Langage compilé (C, C++, Java) 9
3.3.2 Langage interprété (Python, JavaScript, PHP) 9

3.4 Les variables en Python . . . 9
3.4.1 Déclaration et affectation . . . 9
3.4.2 Types de base en Python . . . 10
3.4.3 Opérateur d'affectation . . . 10
3.5 Typage dynamique et représentation mémoire . . . 10
3.6 Lire et afficher des variables en Python . . . 10
3.6.1 Fonction `print()` . . . 10
3.6.2 Fonction `input()` . . . 10
3.6.3 Conversion de types . . . 10
3.7 Opérateurs arithmétiques en Python . . . 11
3.7.1 Opérateurs binaires . . . 11
3.7.2 Opérateurs unaires . . . 11
3.7.3 Fonctions mathématiques . . . 11
3.8 Expressions booléennes en Python . . . 11
3.8.1 Type Booléen . . . 11
3.8.2 Opérateurs de comparaison . . . 11
3.8.3 Opérateurs logiques . . . 12
3.9 Opérateurs sur les chaînes et séquences . . . 12
3.9.1 Concaténation . . . 12
3.9.2 Répétition . . . 12
3.9.3 Comparaison . . . 12
3.9.4 Longueur . . . 12
3.9.5 Appartenance . . . 12
3.9.6 Accès aux éléments . . . 12
3.10 Priorité d'évaluation des expressions . . . 13
3.11 Instructions conditionnelles en Python . . . 13
3.12 Les boucles en Python . . . 13
3.12.1 Boucle `while` . . . 13
3.12.2 Boucle `for` . . . 14
3.12.3 Fonction `range()` . . . 14
3.13 Erreurs de syntaxe vs erreurs de logique . . . 14
3.13.1 Erreur de syntaxe . . . 14
3.13.2 Erreur de logique . . . 15

4 Module 4 : Les fonctions et les modules 16
4.1 Écrire une fonction en Python . . . 16
4.1.1 Structure d'une fonction . . . 16
4.1.2 Exemple complet . . . 16
4.2 Composantes d'une fonction . . . 16
4.2.1 Paramètres . . . 16
4.2.2 Traitement . . . 16
4.2.3 Valeur de retour . . . 16
4.3 Docstrings . . . 17
4.4 Exécution d'une fonction . . . 17
4.4.1 Passage des paramètres . . . 17
4.4.2 Variables locales . . . 17
4.5 Décomposition fonctionnelle . . . 17
4.5.1 Pourquoi utiliser des fonctions ? . . . 17

4.5.2 Fonction comme boîte noire . . . 18
4.6 Les modules . . . 18
4.6.1 Pourquoi utiliser des modules ? . . . 18
4.6.2 Organisation des modules . . . 18
4.6.3 Importation de modules . . . 18
4.7 Tests unitaires . . . 19
4.7.1 Pourquoi des tests unitaires ? . . . 19
4.7.2 Tests avec `assert` . . . 19
4.7.3 Structure d'un test . . . 19

5 Module 5 : Types de données complexes 20
5.1 Notions importantes . . . 20
5.1.1 Qu'est-ce qu'une méthode ? . . . 20
5.1.2 Qu'est-ce qu'une référence ? . . . 20
5.1.3 Mutable vs Immuable . . . 20
5.2 Les chaînes de caractères (classe `str`) . . . 20
5.2.1 Type immuable . . . 20
5.2.2 Accès avec index . . . 20
5.2.3 Conversion de types . . . 21
5.2.4 Caractère de retour chariot . . . 21
5.2.5 Méthode `split()` . . . 21
5.2.6 Méthodes de vérification . . . 21
5.3 Les listes (classe `list`) . . . 21
5.3.1 Définition d'une liste . . . 21
5.3.2 Type hétérogène . . . 22
5.3.3 Type mutable . . . 22
5.3.4 Accès aux éléments . . . 22
5.3.5 Ajout d'éléments . . . 22
5.3.6 Retrait d'éléments . . . 22
5.3.7 Concaténation de listes . . . 23
5.3.8 Opérateurs `for` et `in` . . . 23
5.3.9 Fonction `len()` . . . 23

6 Exemples de code importants 24
6.1 Fonction de recherche du maximum . . . 24
6.2 Fonction de comptage . . . 24
6.3 Fonction de filtrage . . . 24
6.4 Boucles imbriquées . . . 25

7 Conseils pour l'examen 26
7.1 Pièges à éviter . . . 26
7.2 Vérifications avant de remettre . . . 26

1 Module 1 : Présentation de la programmation

1.1 Algorithme

1.1.1 Définition

Un **algorithme** est une séquence finie et non ambiguë d'instructions permettant de résoudre un problème ou d'accomplir une tâche spécifique.

Caractéristiques d'un bon algorithme :

— **Précis** : chaque instruction est clairement définie
— **Déterministe** : mêmes entrées ⇒ mêmes sorties
— **Fini** : se termine après un nombre fini d'étapes
— **Général** : résout une classe de problèmes

1.1.2 Distinction entre algorithme, pseudo-code et programme

Algorithme	**Pseudo-code**	**Programme**
Concept abstrait de résolution de problème	Représentation textuelle structurée de l'algorithme	Implémentation concrète dans un langage de programmation
Indépendant de tout langage	Proche du langage naturel mais structuré	Syntaxe stricte d'un langage spécifique
Ex : "Trier une liste"	Ex : "Pour chaque élément faire..."	Ex : `sorted(list)` en Python

2 Module 2 : Les algorithmes et les instructions procédurales

2.1 Pseudo-code

Le pseudo-code est un langage intermédiaire entre le langage naturel et un langage de programmation. Il permet de décrire un algorithme sans se soucier de la syntaxe d'un langage spécifique.

Exemple :

```
Algorithme Maximum
Entr es: liste de nombres L
Sortie: le plus grand nombre de L

Variables:
    max: nombre
    i: entier

D but
    max <- L[0]
    Pour i de 1    longueur(L)-1 faire
        Si L[i] > max alors
            max <- L[i]
        Fin Si
    Fin Pour
    Retourner max
Fin
```

2.2 Trace d'un algorithme

Faire la trace d'un algorithme consiste à suivre son exécution étape par étape en notant l'évolution des variables.

Exemple de trace :

Étape	i	max	Action
Initial	-	5	max ← L[0]
1	1	5	L[1]=3, 3≯5
2	2	8	L[2]=8, 8>5, max←8
3	3	8	L[3]=2, 2≯8

2.3 Les variables en programmation

2.3.1 Caractéristiques d'une variable

Une variable possède quatre caractéristiques fondamentales :

1. **Nom** : identifiant unique pour accéder à la variable
2. **Type** : nature des données (entier, réel, booléen, etc.)
3. **Valeur** : contenu actuel de la variable
4. **Espace mémoire** : adresse et taille en mémoire

2.3.2 Types de base

- **Booléen** : `Vrai` ou `Faux` (1 bit)
- **Entier** : nombres entiers relatifs (ex : -5, 0, 42)
- **Réel** : nombres à virgule flottante (ex : 3.14, -2.5)
- **Chaîne de caractères** : séquence de caractères (ex : "bonjour")
- **Séquences** : listes, tuples, tableaux

2.4 L'opération d'affectation

L'affectation assigne une valeur à une variable :

$$\texttt{variable} \leftarrow \texttt{expression}$$

Exemples :

```
x <- 5          # x prend la valeur 5
y <- x + 3      # y prend la valeur 8
nom <- "Alice"  # nom prend la valeur "Alice"
```

2.5 Entrées et sorties

2.5.1 Entrées (au clavier)

Demander une chaîne de caractères à l'utilisateur :

```
nom <- LireCha ne()
age <- LireEntier()
```

2.5.2 Sorties (à l'écran)

Afficher une expression :

```
Afficher("Bonjour")
Afficher(x + 5)
Afficher("La valeur est: ", x)
```

2.6 Opérateurs arithmétiques

2.6.1 Opérateurs binaires

- `+` : addition
- `-` : soustraction
- `*` : multiplication
- `/` : division

2.6.2 Opérateurs unaires

- `+x` : signe positif (valeur absolue)
- `-x` : opposé de x

2.7 Les expressions booléennes

2.7.1 Type Booléen

Le type booléen ne comporte que deux valeurs : `Vrai` (True) et `Faux` (False).

2.7.2 Opérateurs de comparaison

— `==` : égal à
— `!=` : différent de
— `>` : plus grand que
— `>=` : plus grand ou égal à
— `<` : plus petit que
— `<=` : plus petit ou égal à

2.7.3 Opérateurs logiques

— **et** (AND) : vrai si les deux opérandes sont vrais
— **ou** (OR) : vrai si au moins un opérande est vrai
— **neg** (NOT) : inverse la valeur de vérité

Tables de vérité :

A	B	A et B	A ou B	neg A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

2.8 Instructions conditionnelles

Structure si-alors-sinon :

```
Si condition alors
    # Instructions si condition est vraie
    instruction1
    instruction2
Sinon
    # Instructions si condition est fausse
    instruction3
Fin Si
```

2.9 Les boucles

2.9.1 Boucle « tant que »

Exécutée tant que la condition est vraie :

```
Tant que condition faire
    # Corps de la boucle
    instruction
Fin Tant que
```

Exemple :

```
i <- 0
Tant que i < 5 faire
    Afficher(i)
    i <- i + 1
Fin Tant que
```

2.9.2 Boucle « pour tout »

Itère sur tous les éléments d'une séquence :

```
Pour chaque element dans sequence faire
    # Corps de la boucle
    instruction
Fin Pour
```

Exemple :

```
Pour i de 1    10 faire
    Afficher(i * 2)
Fin Pour
```

3 Module 3 : Les bases de la programmation et du langage Python

3.1 Rôle de l'interpréteur

L'**interpréteur** Python est un programme qui :
— Lit le code source ligne par ligne
— Traduit chaque instruction en langage machine
— Exécute immédiatement l'instruction traduite
— Gère les erreurs à l'exécution
Fonctionnement :

$$\text{Code source} \xrightarrow{\text{interpréteur}} \text{Analyse} \rightarrow \text{Exécution immédiate}$$

3.2 Code source versus code machine

Code source	Code machine
Écrit par le programmeur	Généré par l'interpréteur/compilateur
Lisible par un humain	Lisible uniquement par la machine
Ex : `print("Bonjour")`	Ex : `01001000 01100101...`
Portable entre plateformes	Spécifique à l'architecture

3.3 Langage interprété vs compilé

3.3.1 Langage compilé (C, C++, Java)

— Traduction complète avant exécution
— Crée un fichier exécutable
— Plus rapide à l'exécution
— Moins portable (dépend de l'OS)

3.3.2 Langage interprété (Python, JavaScript, PHP)

— Traduction ligne par ligne pendant l'exécution
— Nécessite l'interpréteur
— Plus lent mais plus flexible
— Plus portable

3.4 Les variables en Python

3.4.1 Déclaration et affectation

En Python, pas besoin de déclarer le type :

```
x = 5            # x est un entier
nom = "Alice"    # nom est une cha ne
pi = 3.14        # pi est un flottant
```

3.4.2 Types de base en Python

- `bool` : `True` ou `False`
- `int` : nombres entiers
- `float` : nombres décimaux
- `str` : chaînes de caractères
- `tuple` : séquences immuables

3.4.3 Opérateur d'affectation

Important : Ne pas confondre = (affectation) et == (comparaison) !

```
x = 5          # Affectation : x prend la valeur 5
if x == 5:     # Comparaison : teste si x est  gal     5
    print("x vaut 5")
```

3.5 Typage dynamique et représentation mémoire

Python utilise le **typage dynamique** : le type est déterminé à l'exécution.

```
x = 5          # x r f rence un objet entier
x = "hello"    # x r f rence maintenant un objet cha ne
```

Pourquoi l'interpréteur doit connaître le type ?

- Pour allouer la bonne quantité de mémoire
- Pour savoir quelles opérations sont permises
- Pour appeler les bonnes méthodes

3.6 Lire et afficher des variables en Python

3.6.1 Fonction `print()`

```
print("Bonjour")              # Affiche une cha ne
print(x, y, z)                # Affiche plusieurs variables
print("Valeur:", x)           # Combine texte et variable
```

3.6.2 Fonction `input()`

```
nom = input("Entrez votre nom: ")
age = input("Entrez votre  ge : ")  # Retourne une cha ne
```

3.6.3 Conversion de types

```
age = int(input(" ge : "))       # Convertit en entier
prix = float(input("Prix: "))    # Convertit en flottant
texte = str(123)                 # Convertit en cha ne
```

3.7 Opérateurs arithmétiques en Python

3.7.1 Opérateurs binaires

```
+   # Addition
-   # Soustraction
*   # Multiplication
/   # Division (r sultat flottant)
//  # Division enti re
%   # Modulo (reste de la division)
**  # Exposant (puissance)
```

Exemples :

```
7 / 2   # 3.5
7 // 2  # 3
7 % 2   # 1
2 ** 3  # 8
```

3.7.2 Opérateurs unaires

```
-x   # Oppos  de x
+x   # Identique    x
```

3.7.3 Fonctions mathématiques

```
abs(x)    # Valeur absolue
int(x)    # Conversion en entier
float(x)  # Conversion en flottant
```

3.8 Expressions booléennes en Python

3.8.1 Type Booléen

```
vrai = True
faux = False
```

3.8.2 Opérateurs de comparaison

```
==   #  gal
!=   # Diff rent de
>    # Plus grand que
>=   # Plus grand ou  gal
<    # Plus petit que
<=   # Plus petit ou  gal
```

3.8.3 Opérateurs logiques

```
and  # ET logique
or   # OU logique
not  # NON logique
```

Exemples :

```
(x > 0) and (x < 10)    # x entre 0 et 10
(x == 0) or (x == 1)    # x vaut 0 ou 1
not (x > 5)             # x n'est pas > 5
```

3.9 Opérateurs sur les chaînes et séquences

3.9.1 Concaténation

```
"Bonjour" + " " + "Monde"  # "Bonjour Monde"
[1, 2] + [3, 4]            # [1, 2, 3, 4]
```

3.9.2 Répétition

```
"ab" * 3         # "ababab"
[0] * 5          # [0, 0, 0, 0, 0]
```

3.9.3 Comparaison

```
"abc" == "abc"     # True
"abc" != "def"     # True
"abc" < "abd"      # True (ordre lexicographique)
```

3.9.4 Longueur

```
len("hello")    # 5
len([1,2,3])    # 3
```

3.9.5 Appartenance

```
"a" in "abc"        # True
5 in [1, 2, 3]      # False
"x" not in "abc"    # True
```

3.9.6 Accès aux éléments

```
texte = "Python"
texte[0]      # 'P' (premier caract re)
texte[-1]     # 'n' (dernier caract re)
texte[0:3]    # 'Pyt' (tranche)
```

3.10 Priorité d'évaluation des expressions

Ordre de priorité (du plus fort au plus faible) :

1. Parenthèses : `(expression)`
2. Fonctions et méthodes, accès aux éléments
3. Exposant : `**`
4. Multiplication/Division/Modulo : `*, /, //, %`
5. Addition/Soustraction : `+, -`
6. Comparaisons : `==, !=, <, >, <=, >=`
7. `not`
8. `and`
9. `or`
10. Affectation : `=`

Exemple :

```
resultat = 2 + 3 * 4 ** 2  # 2 + (3 * (4 ** 2)) = 50
```

3.11 Instructions conditionnelles en Python

```
if condition1:
    # Instructions si condition1 est vraie
    instruction1
elif condition2:
    # Instructions si condition2 est vraie
    instruction2
else:
    # Instructions si toutes les conditions sont fausses
    instruction3
```

Exemple :

```
note = 85

if note >= 90:
    mention = "A"
elif note >= 80:
    mention = "B"
elif note >= 70:
    mention = "C"
else:
    mention = " chec "
```

3.12 Les boucles en Python

3.12.1 Boucle `while`

```python
while condition:
    # Corps de la boucle
    instruction

# Exemple
i = 0
while i < 5:
    print(i)
    i = i + 1
```

3.12.2 Boucle `for`

```python
# It rer sur une s quence
for element in sequence:
    instruction

# Exemple avec une liste
for nombre in [1, 2, 3, 4]:
    print(nombre * 2)

# Exemple avec une cha ne
for lettre in "Python":
    print(lettre)
```

3.12.3 Fonction `range()`

```python
range(b)        # (0, 1, 2, ..., b-1)
range(a, b)     # (a, a+1, ..., b-1)
range(a, b, p)  # (a, a+p, a+2p, ..., < b)

# Exemples
for i in range(5):
    print(i)     # Affiche 0, 1, 2, 3, 4

for i in range(2, 6):
    print(i)     # Affiche 2, 3, 4, 5
```

3.13 Erreurs de syntaxe vs erreurs de logique

3.13.1 Erreur de syntaxe

Détectée par l'interpréteur avant l'exécution. Le programme ne peut pas s'exécuter.

```python
# Exemples d'erreurs de syntaxe
a + 1 = b           # SyntaxError: cannot assign to expression
print('allo'       # SyntaxError: '(' was never closed
if x > 5            # SyntaxError: expected ':'
```

3.13.2 Erreur de logique

Le programme s'exécute mais produit un résultat incorrect.

```
# Exemple d'erreur de logique
i = 0
while i < 5:
    i == i + 1  # Erreur: == au lieu de =
                # Boucle infinie car i ne change pas
```

4 Module 4 : Les fonctions et les modules

4.1 Écrire une fonction en Python

4.1.1 Structure d'une fonction

```
def nom_fonction(parametre1, parametre2, ...):
    """Docstring: description de la fonction"""
    # Corps de la fonction
    # Traitement des param tres
    resultat = traitement

    # Valeur de retour
    return resultat
```

4.1.2 Exemple complet

```
def addition(a, b):
    """
    Retourne la somme de deux nombres.

    Param tres:
        a (int ou float): premier nombre
        b (int ou float): deuxi me nombre

    Retourne:
        int ou float: la somme de a et b
    """
    somme = a + b
    return somme

# Appel de la fonction
resultat = addition(3, 5)  # resultat = 8
```

4.2 Composantes d'une fonction

4.2.1 Paramètres

Variables locales qui reçoivent les valeurs passées lors de l'appel.

4.2.2 Traitement

Instructions qui transforment les paramètres pour produire un résultat.

4.2.3 Valeur de retour

Résultat renvoyé par l'instruction `return`.

4.3 Docstrings

Documentation intégrée au code :

```
def ma_fonction(x):
    """
    Description courte de la fonction.

    Description d taill e expliquant:
    - Le but de la fonction
    - Les param tres attendus
    - La valeur retourn e
    - Les exceptions possibles

    Exemples:
    >>> ma_fonction(5)
    25
    """
    return x ** 2
```

Pour accéder à la docstring :

```
help(ma_fonction)
ma_fonction.__doc__
```

4.4 Exécution d'une fonction

4.4.1 Passage des paramètres

```
def carre(x):
    return x ** 2

resultat = carre(5)
# 1. La valeur 5 est pass e au param tre x
# 2. Le corps de la fonction est ex cut
# 3. La valeur 25 est retourn e
# 4. resultat re oit la valeur 25
```

4.4.2 Variables locales

Les variables définies dans une fonction n'existent qu'à l'intérieur de celle-ci :

```
def exemple():
    x = 10  # x est locale    exemple()
    return x

print(x)  # Erreur: NameError (x n'existe pas ici)
```

4.5 Décomposition fonctionnelle

4.5.1 Pourquoi utiliser des fonctions ?

— **Réutilisabilité** : écrire une fois, utiliser plusieurs fois

— **Lisibilité** : code plus clair et organisé
— **Maintenabilité** : modifications localisées
— **Abstraction** : cacher la complexité
— **Tests** : tester des unités indépendantes

4.5.2 Fonction comme boîte noire

On utilise une fonction par son interface (paramètres → retour) sans connaître son implémentation interne.

```
# On utilise len() sans savoir comment c'est impl ment
longueur = len("bonjour")  # Retourne 7
```

4.6 Les modules

4.6.1 Pourquoi utiliser des modules ?

— Organisation du code en fichiers séparés
— Réutilisation de code entre programmes
— Partage de fonctionnalités
— Éviter la duplication de code

4.6.2 Organisation des modules

```
mon_projet/
    main.py
    utils.py
    calculs.py
```

4.6.3 Importation de modules

Méthode 1 : import complet

```
import math

x = math.sqrt(16)  # Utilise le pr fixe du module
```

Méthode 2 : from ... import

```
from math import sqrt, pi

x = sqrt(16)  # Acc s direct
print(pi)
```

Méthode 3 : import avec alias

```
import math as m

x = m.sqrt(16)
```

4.7 Tests unitaires

4.7.1 Pourquoi des tests unitaires ?

— Vérifier le bon fonctionnement du code
— Détecter les régressions
— Documenter le comportement attendu
— Faciliter le débogage

4.7.2 Tests avec `assert`

```
def addition(a, b):
    return a + b

# Tests unitaires
assert addition(2, 3) == 5
assert addition(-1, 1) == 0
assert addition(0, 0) == 0
assert addition(2.5, 3.1) == 5.6

print("Tous les tests sont pass s!")
```

4.7.3 Structure d'un test

```
# 1. Pr paration (donn es de test)
entree1 = 5
entree2 = 3
attendu = 8

# 2. Ex cution
resultat = addition(entree1, entree2)

# 3. V rification
assert resultat == attendu, f"Erreur: {resultat} != {attendu}"
```

5 Module 5 : Types de données complexes

5.1 Notions importantes

5.1.1 Qu'est-ce qu'une méthode ?

Une **méthode** est une fonction associée à un objet. Elle s'appelle avec la syntaxe :

```
objet.methode(arguments)
```

Exemple :

```
texte = "bonjour"
texte.upper()  # M thode upper() de la classe str
# Retourne "BONJOUR"
```

5.1.2 Qu'est-ce qu'une référence ?

Une variable contient une **référence** (adresse mémoire) vers un objet, pas l'objet lui-même.

```
a = [1, 2, 3]
b = a  # b r f rence le M ME objet que a

b.append(4)
print(a)  # [1, 2, 3, 4] - a est modifi !
```

5.1.3 Mutable vs Immuable

Type mutable : Peut être modifié après sa création.

```
liste = [1, 2, 3]
liste[0] = 10  # Modification possible
print(liste)   # [10, 2, 3]
```

Type immuable : Ne peut pas être modifié après sa création.

```
texte = "bonjour"
# texte[0] = "B"  # Erreur! TypeError
texte = "Bonjour"  # Cr e un NOUVEL objet
```

5.2 Les chaînes de caractères (classe `str`)

5.2.1 Type immuable

Les chaînes de caractères sont immuables : on ne peut pas modifier un caractère individuel.

5.2.2 Accès avec index

```
texte = "Python"

texte[0]     # 'P' (premier caract re)
texte[5]     # 'n' (dernier caract re)
```

```
texte[-1]    # 'n' (index n gatif)
texte[0:3]   # 'Pyt' (tranche)
texte[2:]    # 'thon' (de l'index 2    la fin)
texte[:3]    # 'Pyt' (du d but    l'index 3)
```

5.2.3 Conversion de types

```
int("314")         # 314
float("3.14")      # 3.14
str(3.14)          # "3.14"
int(3.14)          # 3 (troncature)
float(314)         # 314.0

# Attention aux erreurs
# int("3.14")      # ValueError!
int(float("3.14")) # 3 (conversion en deux   tapes )
```

5.2.4 Caractère de retour chariot

```
texte = "ligne1\nligne2"
print(texte)
# Affiche:
# ligne1
# ligne2
```

5.2.5 Méthode `split()`

Divise une chaîne en liste selon un séparateur :

```
"a,b,c".split(",")       # ["a", "b", "c"]
"1 2 3".split()          # ["1", "2", "3"] (espace par d faut)
"a;b;c".split(";")       # ["a", "b", "c"]
```

5.2.6 Méthodes de vérification

```
"ABC".isupper()     # True
"abc".islower()     # True
"123".isdigit()     # True
"ABC".islower()     # False
"abc123".isdigit() # False (contient des lettres)
```

5.3 Les listes (classe `list`)

5.3.1 Définition d'une liste

```
# Liste vide
liste_vide = []

```

```
# Liste d'entiers
nombres = [1, 2, 3, 4, 5]

# Liste de chaînes
fruits = ["pomme", "banane", "orange"]

# Liste hétérogène (différents types)
mixte = [1, "deux", 3.0, True]

# Liste de listes
matrice = [[1, 2], [3, 4]]
```

5.3.2 Type hétérogène

Une liste peut contenir des éléments de types différents :

```
liste = [1, "texte", 3.14, [1, 2], True]
```

5.3.3 Type mutable

Les listes sont mutables : on peut les modifier après création.

5.3.4 Accès aux éléments

```
liste = ["a", "b", "c", "d"]

liste[0]      # "a" (premier élément)
liste[-1]     # "d" (dernier élément)
liste[1:3]    # ["b", "c"] (tranche)
```

5.3.5 Ajout d'éléments

Méthode `append()` : Ajoute à la fin

```
liste = [1, 2, 3]
liste.append(4)
print(liste)  # [1, 2, 3, 4]
```

Méthode `insert()` : Insère à une position

```
liste = [1, 2, 4]
liste.insert(2, 3)  # Insère 3 à l'index 2
print(liste)        # [1, 2, 3, 4]
```

5.3.6 Retrait d'éléments

Méthode `pop()` : Retire et retourne un élément

```
liste = [1, 2, 3, 4]
element = liste.pop()       # Retire le dernier (4)
print(liste)                # [1, 2, 3]

element = liste.pop(1)      # Retire l'élément à l'index 1
print(liste)                # [1, 3]
```

5.3.7 Concaténation de listes

```
liste1 = [1, 2]
liste2 = [3, 4]
liste3 = liste1 + liste2
print(liste3)  # [1, 2, 3, 4]
```

5.3.8 Opérateurs `for` et `in`

```
# It ration sur une liste
for element in [1, 2, 3]:
    print(element * 2)

# Test d'appartenance
5 in [1, 2, 3, 4, 5]      # True
10 in [1, 2, 3]           # False
"a" in ["a", "b", "c"]    # True
```

5.3.9 Fonction `len()`

```
len([1, 2, 3])            # 3
len([])                   # 0
len(["a", "b", "c", "d"]) # 4
```

6 Exemples de code importants

6.1 Fonction de recherche du maximum

```python
def trouver_maximum(liste):
    """Retourne le maximum d'une liste de nombres."""
    if len(liste) == 0:
        return None

    maximum = liste[0]
    for element in liste:
        if element > maximum:
            maximum = element

    return maximum

# Tests
assert trouver_maximum([1, 5, 3, 9, 2]) == 9
assert trouver_maximum([-1, -5, -3]) == -1
```

6.2 Fonction de comptage

```python
def compter_occurrences(liste, element):
    """Compte le nombre d'occurrences d'un    lment    dans une liste
   ."""
    compteur = 0
    for item in liste:
        if item == element:
            compteur = compteur + 1
    return compteur

# Tests
assert compter_occurrences([1, 2, 3, 2, 4, 2], 2) == 3
assert compter_occurrences(["a", "b", "a"], "a") == 2
```

6.3 Fonction de filtrage

```python
def filtrer_positifs(liste):
    """Retourne une nouvelle liste avec seulement les nombres
   positifs."""
    resultat = []
    for nombre in liste:
        if nombre > 0:
            resultat.append(nombre)
    return resultat

# Tests
assert filtrer_positifs([1, -2, 3, -4, 5]) == [1, 3, 5]
assert filtrer_positifs([-1, -2, -3]) == []
```

6.4 Boucles imbriquées

```python
# Table de multiplication
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()  # Nouvelle ligne

# Affiche:
# 1 2 3 4 5
# 2 4 6 8 10
# 3 6 9 12  15
# 4 8 12  16  20
# 5 10  15  20  25
```

7 Conseils pour l'examen

— **Comprendre les concepts** : Ne pas juste mémoriser, comprendre pourquoi
— **Pratiquer la trace** : Savoir suivre l'exécution d'un algorithme
— **Lire attentivement** : Bien comprendre les questions
— **Gérer son temps** : Commencer par les questions faciles
— **Tester mentalement** : Vérifier votre code avec des exemples simples
— **Respecter la syntaxe** : Attention aux deux-points, indentation, parenthèses

7.1 Pièges à éviter

1. Confondre `=` (affectation) et `==` (comparaison)
2. Oublier l'indentation dans les blocs
3. Index hors limites (rappel : commence à 0)
4. Modifier une liste pendant l'itération
5. Oublier le `return` dans une fonction
6. Confondre mutable et immuable

7.2 Vérifications avant de remettre

— Le code est-il syntaxiquement correct ?
— Les cas limites sont-ils gérés (liste vide, zéro, négatif) ?
— Les tests unitaires passent-ils ?
— La réponse correspond-elle à la question posée ?